SEN4100 – Software Project Management

Week 6 Lecture Notes: Project Scheduling & Risk Management

Haliç University, Faculty of Engineering, Department of Software Engineering

Course Information & Academic Framework

- **Course Learning Outcomes (CLOs) Addressed:**
  - Apply scheduling, budgeting, and resource-allocation techniques.
  - Analyze and manage software risks systematically.
- **Primary Textbooks:** Pressman & Maxim (*Software Engineering*), Hughes & Cotterell (*Software Project Management*).

  Note on Schedule: This is our final lecture before the Midterm Examination. It synthesizes the effort estimates from Week 5 into a chronological schedule and applies systematic risk management to defend that schedule against uncertainties.

PART I: PROJECT SCHEDULING (GANTT, CPM, PERT)

In previous weeks, the WBS defined *what* tasks to do, and Estimation defined *how long* they take. Scheduling determines *when* they will be executed and in *what sequence*.

1. Activity Sequencing & Precedence Dependencies

Tasks in a software project cannot be executed randomly; they are bound by logical dependencies. The **Precedence Diagramming Method (PDM)** defines four types of logical relationships between tasks:

1. **Finish-to-Start (FS):** The most common dependency. Task A must finish before Task B can start. *(e.g., You must finish designing the Database Schema [A] before you can start writing the SQL Migration Scripts [B]).*
2. **Start-to-Start (SS):** Task B cannot start until Task A has started. *(e.g., Code Review [B] can start once Implementation [A] has started).*
3. **Finish-to-Finish (FF):** Task B cannot finish until Task A has finished. *(e.g., User Documentation [B] cannot be finalized until Software Testing [A] is finished).*
4. **Start-to-Finish (SF):** Task B cannot finish until Task A has started (Rare in software engineering).

2. Network Diagramming & The Critical Path Method (CPM)

The **Activity-on-Node (AON)** diagram represents the project as a network, where boxes (nodes) represent tasks and arrows represent dependencies.

The **Critical Path Method (CPM)** is a deterministic scheduling algorithm used to calculate the absolute minimum total time required to complete the project.

Key CPM Calculations:

- **Forward Pass (Calculates Early Dates):**
  - **Early Start (ES):** The earliest time a task can begin.
  - **Early Finish (EF):** $ES + Duration$.
- **Backward Pass (Calculates Late Dates):**
  - **Late Finish (LF):** The latest time a task can finish without delaying the entire project.
  - **Late Start (LS):** $LF - Duration$.
- **Slack / Float Calculation:**
  - **Slack = LS - ES** (or LF - EF).
  - Slack represents how much a task can be delayed without delaying the final project deadline.

The Critical Path: The longest contiguous path of dependent activities through the network. **Tasks on the Critical Path have exactly ZERO Slack.** If a critical path task is delayed by 1 day, the entire software release is delayed by 1 day.

3. Program Evaluation and Review Technique (PERT)

While CPM uses fixed (deterministic) durations, **PERT** uses probabilistic durations to account for estimation uncertainty. It requires developers to provide three estimates for each task:

1. **Optimistic (O):** Everything goes perfectly.
2. **Most Likely (M):** Normal execution.
3. **Pessimistic (P):** Everything goes wrong.

The PERT Expected Duration (t_e) Formula:

$$t_e = \frac{O + 4M + P}{6}$$

Managerial Value: This weighted average heavily favors the "Most Likely" estimate but mathematically builds a buffer for worst-case scenarios, producing a much safer schedule than single-point estimates.

4. Gantt Charts

Developed by Henry Gantt, the Gantt Chart is a bar chart that illustrates the project schedule.

- **Function:** While CPM/AON diagrams are excellent for mathematical calculation by the SPM, Gantt charts are the standard visual tool for reporting chronological progress to executive stakeholders and clients.

PART II: RISK IDENTIFICATION & MANAGEMENT

A project schedule is merely a hypothesis. **Risk Management** is the discipline of defending that hypothesis against reality.

5. Software Risk Taxonomy

A risk is an uncertain event or condition that, if it occurs, has a positive or negative effect on project objectives. Risks are categorized into a taxonomy:

- **Technical Risks:** Unproven architectures, API deprecation, severe legacy technical debt, or failure to meet scalability NFRs.
- **Project Management Risks:** Inaccurate effort estimation, poor schedule sequencing, or inadequate resource allocation.
- **Organizational Risks:** Loss of executive sponsorship, shifting corporate strategy, or budget cuts.
- **External/Environmental Risks:** Changes in regulatory laws (e.g., new GDPR mandates), vendor bankruptcy, or macroeconomic shifts.

6. Qualitative Assessment: The Probability-Impact Matrix

Not all risks require equal attention. The SPM assesses risks across two dimensions:

1. **Probability (P):** The likelihood of the risk occurring (e.g., 0.1 to 0.9).
2. **Impact (I):** The severity of the consequence if it occurs (e.g., Financial loss, schedule delay).

Risk Exposure (RE) = Probability × Impact

Risks are then plotted on a **Probability-Impact Matrix**:

- **Red Zone (High P / High I):** Critical threats. Require immediate, aggressive mitigation plans.
- **Yellow Zone (Moderate):** Monitored risks. Trigger conditions are established.
- **Green Zone (Low P / Low I):** Accepted risks. Kept on a watchlist with minimal active management.

7. Risk Response Strategies

Once risks are prioritized, the SPM must assign a concrete response strategy:

1. **Mitigation (Azaltma):** Taking proactive steps to reduce the probability or impact of a threat (e.g., conducting an architectural spike to test an unproven API before building the whole system).
2. **Avoidance (Kaçınma):** Changing the project plan to eliminate the threat entirely (e.g., dropping a highly complex, non-essential feature from the scope).
3. **Transfer (Devretme):** Shifting the financial or operational impact to a third party (e.g., purchasing cyber-liability insurance, or outsourcing the payment gateway to Stripe instead of building it internally).
4. **Acceptance (Kabul Etme):** Acknowledging the risk but taking no proactive action, usually because the cost of mitigation exceeds the cost of the impact. A contingency reserve (budget buffer) is established.

8. The Risk Register

The **Risk Register** is the central governance document for risk management. It must include:

- Risk ID and Description.
- Category (Technical, Org, etc.).
- Calculated Probability, Impact, and Risk Exposure score.
- Selected Response Strategy (Mitigation, Transfer, etc.).
- **Assigned Risk Owner** (The specific engineer or manager responsible for monitoring the trigger).

9. Alignment with Term Project: Milestone 2 & Midterm

This lecture completes the theoretical foundation for the first half of the semester.

Midterm Examination (Week 7) Preparation:

- The exam will cover all concepts from Weeks 1 through 6.
- Expect applied questions: You will be asked to draw an AON network diagram, calculate the Critical Path (ES, EF, LS, LF, Slack), and calculate PERT durations.
- You must know how to identify risks and assign them to a Probability-Impact Matrix.

Milestone 2 Deliverable (Due post-midterm):

- **Scheduling:** Your squad must translate your WBS (Milestone 1) into a baseline schedule, assigning task dependencies and identifying your project's Critical Path.
- **Risk Register:** You must submit a formal Risk Register identifying at least 5 critical risks specific to your software project, complete with mitigation strategies and assigned owners.

10. Classroom Discussion Scenarios

1. **The "Impossible Deadline" Dilemma:**
   - After calculating your CPM network, you determine the absolute Critical Path duration is 14 weeks. The Executive Sponsor informs you the product *must* launch in 10 weeks for a trade show.
   - *Question:* What are your only two mathematically viable options? (Hint: *Crashing* the schedule by adding resources, which increases cost, or *Fast-Tracking* by running sequential tasks in parallel, which increases risk).
2. **Risk vs. Issue:**
   - A developer states in a meeting: *"Our primary risk is that the lead backend engineer quit yesterday."*
   - *Question:* Why is this fundamentally incorrect terminology in project management? (Hint: A risk is an *uncertain future event*. If the engineer has already quit, it is an **Issue**, not a Risk. Issues require immediate resolution, not probability analysis).